

TCSS 487 Cryptography

Practical Project — Cryptographic Library & App — Part 1

Thienan Nguyen

May 2026

1 Overview

This report describes Part 1 of the cryptographic library and application project for TCSS 487. The project is written entirely in Java with no external cryptographic libraries. All symmetric services are built on the SHA-3/SHAKE (Keccak) sponge construction as specified in FIPS 202.

The submission contains three source files:

- `SHA3SHAKE.java` — implementation of the SHA-3/SHAKE sponge.
- `Main.java` — command-line application offering the four required services.
- `App.java` — empty placeholder class (no logic).

2 Attribution

The Keccak-f[1600] permutation (round-constant table `RC`, rho-rotation offsets `ROTC`, and pi lane-permutation table `PILN`) in `SHA3SHAKE.java` is ported, with minor adaptations, from Markku-Juhani O. Saarinen's public-domain C reference implementation *tiny_sha3*:

https://github.com/mjosaarinen/tiny_sha3/blob/master/sha3.c

The sponge wrapper, byte/lane plumbing, padding logic, and the Java public interface are original work.

3 Compiling the Project

All source files are located in the `src/` directory. Open a terminal, navigate to `src/`, and run:

```
javac SHA3SHAKE.java Main.java
```

The program is then invoked as:

```
java Main <command> [arguments...]
```

where `<command>` is one of `hash`, `mac`, `encrypt`, or `decrypt`.

4 Services

4.1 Service 1 — Cryptographic Hash (SHA-3-256 and SHA-3-512)

Command syntax:

```
java Main hash <file>
```

- **<file>** — path to the file to be hashed.

What it does: Reads the entire file into a byte array and computes both the SHA-3-256 and SHA-3-512 hashes using the static `SHA3SHAKE.SHA3()` helper. Both digests are printed to standard output in hexadecimal.

Example:

```
java Main hash test.txt
SHA-3-256: 91468dc1dec55577d293c47a4c092dea71a43c49a5e79064a3e0e055acedc41d
SHA-3-512: 1f6341857b0c8ae0e59e8aff3a2cad25be1f3cd6b339b260425f8bf2585b2876...
```

4.2 Service 2 — Authentication Tag / MAC (SHAKE-128 and SHAKE-256)

Command syntax:

```
java Main mac <file> <passphrase> <tagLengthBytes>
```

- **<file>** — path to the file to authenticate.
- **<passphrase>** — the secret passphrase (UTF-8 string).
- **<tagLengthBytes>** — desired MAC output length in bytes (e.g. 32 for a 256-bit tag).

What it does: Initializes a `SHA3SHAKE` sponge at each security level (128 and 256), absorbs the passphrase followed by the file contents, and squeezes the requested number of bytes. Both tags are printed in hexadecimal.

Example:

```
java Main mac test.txt "mysecret" 32
SHAKE-128 MAC (32 bytes):
    a3ef5eb8a0e417c9b21953fb1ef1fe392f85039bb9945e65d701897610cc329a
SHAKE-256 MAC (32 bytes):
    b26b6ee780da70653d85ee28366c160101844f7b631c9a6aca7051292d86d2b8
```

4.3 Service 3 — Symmetric Encryption

Command syntax:

```
java Main encrypt <inputFile> <outputFile> <passphrase>
```

- **<inputFile>** — path to the plaintext file.
- **<outputFile>** — path where the cryptogram will be written.
- **<passphrase>** — the secret passphrase (UTF-8 string).

What it does:

1. Derives a 128-bit symmetric key: `key = SHAKE-128(passphrase, 128 bits)`.
2. Generates a random 128-bit (16-byte) nonce using `java.security.SecureRandom`.
3. Initializes a SHAKE-128 sponge, absorbs the nonce then the key, and squeezes a keystream equal in length to the plaintext.
4. XORs the keystream with the plaintext to produce the ciphertext.
5. Writes the cryptogram as: **nonce (16 bytes) || ciphertext**.

Example:

```
java Main encrypt test.txt test.enc "mysecret"
Encrypted: test.txt -> test.enc
```

4.4 Service 4 — Symmetric Decryption

Command syntax:

```
java Main decrypt <inputFile> <outputFile> <passphrase>
```

- `<inputFile>` — path to the cryptogram produced by `encrypt`.
- `<outputFile>` — path where the recovered plaintext will be written.
- `<passphrase>` — the same passphrase used during encryption.

What it does:

1. Re-derives the 128-bit symmetric key: `key = SHAKE-128(passphrase, 128 bits)`.
2. Reads the first 16 bytes of the cryptogram as the nonce; the remainder is the ciphertext.
3. Initializes a SHAKE-128 sponge, absorbs the nonce then the key, and squeezes a keystream equal in length to the ciphertext.
4. XORs the keystream with the ciphertext to recover the plaintext.
5. Writes the recovered plaintext to `<outputFile>`.

Example:

```
java Main decrypt test.enc test.dec "mysecret"
Decrypted: test.enc -> test.dec
```

5 Implementation Notes

SHA3SHAKE class

The class models a plain, non-duplexing sponge over the Keccak-f[1600] permutation. The rate and capacity are determined by the `suffix` argument to `init()`:

Mode	Rate (bytes)	Capacity (bytes)
SHAKE-128	168	32
SHA-3-224	144	56
SHA-3-256 / SHAKE-256	136	64
SHA-3-384	104	96
SHA-3-512	72	128

SHA-3 finalization uses domain byte `0x06`; SHAKE finalization uses `0x1F`. The `pad10*1` rule is applied at the end of the absorb phase before the first permutation of the squeeze phase.

Keystream cipher

The stream cipher used for encryption and decryption is SHAKE-128 seeded with the nonce followed by the derived key. Because SHAKE is an extendable-output function, it produces an arbitrarily long pseudorandom stream suitable for use as a stream cipher.

6 Known Bugs and Limitations

- The `mac` command accepts the tag length in *bytes*. Passing a non-integer value will cause a `NumberFormatException`.
- Passphrases containing spaces must be quoted on the command line (e.g. `"my secret"`).
- Decrypting a file not produced by this program's `encrypt` command will produce garbled output without warning, since no MAC verification is included in the non-bonus version.
- Very large files are read entirely into memory; files larger than the available heap will cause an `OutOfMemoryError`.

7 References

- National Institute of Standards and Technology. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. FIPS PUB 202, August 2015. <https://doi.org/10.6028/NIST.FIPS.202>
- M.-J. O. Saarinen. *tiny_sha3* (public-domain C reference implementation). https://github.com/mjosaarinen/tiny_sha3/blob/master/sha3.c